

NYC / AI AGENT SECURITY SUMMIT

Your Agent Escaped Without Escaping the Sandbox

Yossi Eliaz

Principal Engineer and Head of DevRel at Incredibuild.com

October 21, 2026

The morning after

FICTIONAL LAB INCIDENT

sandbox.lifecycle	terminated
resource.limits	respected
host.escape	none observed
task.status	success
private.fixture	copied
release.policy	changed
acceptance.check	replaced

The job was ordinary

**“Fix the parser test.
Upload the report.”**

A repository. A runner. An agent that can execute code.

The runner brought extra authority

Intended task

Convenient extra access

Read source

Parent service credential

Write a patch

Entire project mount

Upload a report

Arbitrary uploads to an allowed service

Run tests

Writable verifier and expected results

Threat model

Agent-controlled: generated code, shell commands, workspace files, outbound request contents.

Trusted: host enforcement, credential broker, external verifier, publication gate.

Assume the agent may act adversarially. No kernel exploit is needed for these scenarios.

01 / INHERITED CREDENTIALS

The runner's identity came along for the ride

The agent can do everything the projected credential permits.

A child process receives a bigger job

ILLUSTRATIVE PYTHON

```
env = os.environ.copy()
env["TASK_ID"] = "parser-fix"
subprocess.run(agent_command, env=env)
```

The task ID describes intent. The inherited token grants authority.

Reference: `Linux environ(7)`

The credential boundary

Launch: explicit environment, closed extra descriptors, isolated process identity.

Service access: broker grants a specific operation on a specific resource.

Lifetime: short expiry, audience checks, revocation where supported.

A short-lived broad token remains broad until it expires.

Reference: RFC 9700, access-token privilege restriction

Proof obligation 01

PLANNED LAB / EXPECTED OUTCOMES

	Before	After
Mock administrative action	Accepted	Denied
Read task input	Accepted	Accepted
Expired task credential	—	Denied

Observe the service decision outside the agent.

A writable mount is a grant of authority

The filesystem behaves exactly as configured.

The workspace includes the answer key

FICTIONAL FIXTURE LAYOUT

```
/work/  
  src/parser.py  
  tests/test_parser.py  
  release/policy.json  
  verifier/check.py
```

A writable project root lets the agent change more than the patch.

Reference: Docker bind mounts

The filesystem boundary

Resource

Agent access

Frozen source input

Read only

Patch and report output

Write within task directory

Release policy

No write access

Runtime control socket

Absent

The host defines the mount set. A path check inside agent code cannot enforce it.

Proof obligation 02

PLANNED LAB / EXPECTED OUTCOMES

Before: a change to the mounted policy fixture reaches the host fixture.

After: the policy stays unchanged. The patch output remains writable.

Compare host-side bytes before and after. Record the actual mount configuration.

**The hostname is allowed.
The recipient is wrong.**

An upload service can serve many accounts.

One host, two destinations

ILLUSTRATIVE REQUESTS TO A MOCK SERVICE

POST /accounts/team/reports

Host: reports.example

Body: approved_report

POST /accounts/other/uploads

Host: reports.example

Body: synthetic_private_fixture

A host allowlist cannot distinguish these two data flows.

The authority and egress boundary

External broker: fixed service, fixed recipient, narrow operation.

Data release: trusted code constructs the report from permitted fields.

Network policy: the agent cannot bypass the broker.

If arbitrary agent-chosen bytes may leave, a secret may leave with them.

Proof obligation 03

PLANNED LAB / EXPECTED OUTCOMES

Unauthorized recipient	Denied
Private fixture in report body	Denied
Direct network bypass	Denied
Approved report to team account	Accepted

The receiving service and network boundary supply the evidence.

04 / VERIFIER TAMPERING

The agent changed what “passed” meant

The dashboard trusts a check the agent can rewrite.

An invalid artifact gets a green result

ILLUSTRATIVE MUTABLE CHECKER

```
# The intended check:  
assert parse("malformed") == REJECT  
  
# The agent-controlled replacement:  
print("PASS")  
raise SystemExit(0)
```

Exit code zero proves only that this process returned zero.

The evidence boundary

Verifier: outside the agent's write and process authority.

Decision: binds artifact digest, verifier identity and policy version.

History: externally anchored records expose later rewriting.

The publication gate accepts only the exact verified artifact.

Reference: SLSA artifact verification

A hash chain needs an external anchor

**An attacker who owns
the whole log can
rewrite the whole chain.**

Keep the checkpoint or signing authority outside the agent.

Proof obligation 04

PLANNED LAB / EXPECTED OUTCOMES

Reject: invalid artifact, modified verifier, stale approval, substituted artifact.

Accept: valid artifact under the pinned verifier and policy.

Verification and publication must refer to the same immutable bytes.

Where enforcement belongs

Boundary

Execution

Filesystem

Credentials and egress

Evidence and acceptance

Authority that enforces it

Host kernel or hypervisor

Host mount and access policy

Broker, service and network policy

External verifier and publication gate

The same job, with bounded authority

Read the input.

Produce the patch.

Request a narrow action.

Submit evidence for verification.

Four questions for your deployment

1. Which host interfaces can this process reach?
2. Which exact bytes can it read and change?
3. Which actions and recipients can it authorize?
4. Can it change the evidence used to accept its work?

THE QUESTION TO TAKE HOME

Who gave this process the authority?

github.com/zozo123/nyc-talk

Yossi Eliaz / Questions

References and scope

Linux environ(7): process environment.

Docker bind mounts: host filesystem access.

RFC 9700: OAuth access-token privilege restriction.

SLSA verification: artifact identity and trusted provenance.

The incident is fictional. Demo outcomes are planned assertions. This deck contains no measured security results.

Discussion: residual risk

A trusted broker can still contain bugs.

A valid signature can authenticate incorrect evidence.

A correct verifier can check an incomplete specification.

Permitted output can still disclose information.

Every boundary needs a named owner and a testable claim.